

Nginx

今日知识点

Nginx 入门

Nginx 反向代理

Nginx 负载均衡

Nginx 动静分离

Nginx 高可用集群

Nginx 原理与优化参数配置

1、Nginx 入门

1.1、Nginx 简介

Nginx (“engine x”) 是一个**高性能的 HTTP 和反向代理服务器**,特点是**占有内存少, 并发能力强, 擅长处理静态资源和高并发**,有报告表明能支持高达 50,000 个并发连接数,而 tomcat 是 web 服务器,擅长处理动态资源。事实上 nginx 的并发能力确实在同类型的网页服务器中表现较好,中国大陆使用 nginx 网站用户有: 百度、京东、新浪、网易、腾讯、淘宝等。

Nginx 官网: <http://nginx.org/>



1.2、Nginx 优势

和其它 web 服务器(tomcat、apache)相比,Nginx 有如下几个优势:

- 高并发
- 热部署
- 快
- 低功耗
- 可反向代理、负载均衡、处理静态资源文件

1.3、Nginx 安装

nginx 是 C 语言开发, 建议在 linux 上运行

1.3.1、安装 nginx 所需要的预处理环境

➤gcc

安装 nginx 需要先将官网下载的源码进行编译,编译依赖 gcc 环境,如果没有 gcc 环境,需要安装 gcc:

```
yum install gcc-c++
```

➤PCRE

PCRE(Perl Compatible Regular Expressions)是一个 Perl 库,包括 perl 兼容的正则表达式库。nginx 的 http 模块使用 pcre 来解析正则表达式,所以需要在 linux 上安装 pcre 库。

```
yum install -y pcre pcre-devel
```

注:pcre-devel 是使用 pcre 开发的一个二次开发库。nginx 也需要此库。

➤zlib

zlib 库提供了很多种压缩和解压缩的方式,nginx 使用 zlib 对 http 包的内容进行 gzip,所以需要在 linux 上安装 zlib 库。

```
yum install -y zlib zlib-devel
```

➤openssl

OpenSSL 是一个强大的安全套接字层密码库,囊括主要的密码算法、常用的密钥和证书封装管理功能及 SSL 协议,并提供丰富的应用程序供测试或其它目的使用。

nginx 不仅支持 http 协议,还支持 https (即在 ssl 协议上传输 http),所以需要在 linux 安装 openssl 库。

```
yum install -y openssl openssl-devel
```

1.3.2、安装 nginx

将 nginx 压缩包拷贝至 linux 服务器,解压 nginx 压缩包。

第一步:解压并进入 nginx 安装目录

```
tar -zxvf nginx-1.12.2.tar.gz -C /opt/install  
cd nginx-1.12.2 .
```

第二步:生成 makefile 文件

./configure --help 查询详细参数 (参考本教程附录部分:nginx 编译参数)

参数设置如下:提示先创建该目录:mkdir -p /var/temp/nginx

```
./configure \  
--prefix=/usr/local/nginx \  
--pid-path=/var/run/nginx/nginx.pid \  
--lock-path=/var/lock/nginx.lock \  
--error-log-path=/var/log/nginx/error.log \  
--http-log-path=/var/log/nginx/access.log \  
--with-http_gzip_static_module \  
--http-client-body-temp-path=/var/temp/nginx/client \  
--http-proxy-temp-path=/var/temp/nginx/proxy \  
--http-fastcgi-temp-path=/var/temp/nginx/fastcgi \  
--http-uwsgi-temp-path=/var/temp/nginx/uwsgi \  
--http-scgi-temp-path=/var/temp/nginx/scgi
```

第三步：编译并安装

make

make install

第四步：复制 nginx 命令为全局命令

将 nginx 命令复制为全局命令之后，就可以在任意地方使用了。

```
cp /usr/local/nginx/sbin/nginx /usr/local/bin/
```

第五步：启动 nginx，并测试

```
cd /usr/local/nginx/sbin/
```

```
./nginx
```

测试一：查询 nginx 进程：

```
[root@server01 sbin]# ps aux|grep nginx
root      15098  0.0  0.0  3568  528 ?        Ss   20:15   0:00 nginx: master process ./nginx
nobody    15099  0.0  0.0  3768  884 ?        S    20:15   0:00 nginx: worker process
```

15098 是 nginx 主进程的进程 id，15099 是 nginx 工作进程的进程 id

注意：执行 ./nginx 启动 nginx，这里可以 -c 指定加载的 nginx 配置文件，如下：

```
./nginx -c /usr/local/nginx/conf/nginx.conf
```

如果不指定 -c，nginx 在启动时默认加载 conf/nginx.conf 文件，此文件的地址也可以在编译安装 nginx 时指定 ./configure 的参数（--conf-path= 指向配置文件（nginx.conf））

测试二：在 windows 上测试 nginx

nginx 安装成功，启动 nginx，即可访问虚拟机上的 nginx：



Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

到这说明 nginx 上安装成功。

查看开放的端口号

```
firewall-cmd --list-all
```

设置开放的端口号

```
firewall-cmd --add-port=80/tcp --permanent
```

重启防火墙

```
firewall-cmd -reload .
```

1.4、Nginx 常用命令

1.4.1、启动 nginx

```
cd /usr/local/nginx/sbin/
```

```
./nginx
```

1.4.2、停止 nginx

方式 1，快速停止：

```
cd /usr/local/nginx/sbin  
./nginx -s stop
```

此方式相当于先查出 nginx 进程 id 再使用 kill 命令强制杀掉进程。

方式 2，完整停止(建议使用)：

```
cd /usr/local/nginx/sbin  
./nginx -s quit
```

此方式停止步骤是待 nginx 进程处理任务完毕进行停止。

1.4.3、重启 nginx

方式 1，先停止再启动：

对 nginx 进行重启相当于先停止 nginx 再启动 nginx，即先执行停止命令再执行启动命令。

如下：

```
./nginx -s quit  
./nginx
```

方式 2，重新加载配置文件：

当 nginx 的配置文件 nginx.conf 修改后，要想让配置生效需要重启 nginx，使用-s reload 不用先停止 nginx 再启动 nginx 即可将配置信息在 nginx 中生效，如下：

```
./nginx -s reload
```

1.5、配置文件 nginx.conf

nginx 安装目录下，其默认的配置文件的都放在这个目录的 conf 目录下，而主配置文件 nginx.conf 也在其中，后续对 nginx 的使用基本上都是对此配置文件进行相应的修改

```

[root@slavel ~]# cd /usr/local/nginx/conf/
[root@slavel conf]# ll
总用量 68
-rw-r--r--. 1 root root 1077 7月 29 21:48 fastcgi.conf
-rw-r--r--. 1 root root 1077 7月 29 21:48 fastcgi.conf.default
-rw-r--r--. 1 root root 1007 7月 29 21:48 fastcgi_params
-rw-r--r--. 1 root root 1007 7月 29 21:48 fastcgi_params.default
-rw-r--r--. 1 root root 2837 7月 29 21:48 koi-utf
-rw-r--r--. 1 root root 2223 7月 29 21:48 koi-win
-rw-r--r--. 1 root root 5170 7月 29 21:48 mime.types
-rw-r--r--. 1 root root 5170 7月 29 21:48 mime.types.default
-rw-r--r--. 1 root root 2656 7月 29 21:48 nginx.conf
-rw-r--r--. 1 root root 2656 7月 29 21:48 nginx.conf.default
-rw-r--r--. 1 root root 636 7月 29 21:48 scgi_params
-rw-r--r--. 1 root root 636 7月 29 21:48 scgi_params.default
-rw-r--r--. 1 root root 664 7月 29 21:48 uwsgi_params
-rw-r--r--. 1 root root 664 7月 29 21:48 uwsgi_params.default
-rw-r--r--. 1 root root 3610 7月 29 21:48 win-utf
[root@slavel conf]#

```

配置文件中有很多#，开头的表示注释内容，我们去掉所有以 # 开头的段落，精简之后的内容如下：

```

worker_processes 1;

events {
    worker_connections 1024;
}

http {
    include mime.types;
    default_type application/octet-stream;

    sendfile on;

    keepalive_timeout 65;

    server {
        listen 80;
        server_name localhost;

        location / {
            root html;
            index index.html index.htm;
        }
    }
}

```

根据上述文件，我们可以很明显的将 nginx.conf 配置文件分为三部分：

1.5.1、第一部分：全局块

从配置文件开始到 events 块之间的内容, 主要会设置一些影响 nginx 服务器整体运行的配置指令, 主要包括配置运行 Nginx 服务器的用户 (组)、允许生成的 worker process 数, 进程 PID 存放路径、日志存放路径和类型以及配置文件的引入等。比如上面第一行配置的:

```
worker_processes 1;
```

这是 Nginx 服务器并发处理服务的关键配置, **worker_processes 值越大, 可以支持的并发处理量也越多**, 但是会受到硬件、软件等设备的制约。

1.5.2、第二部分：events 块

比如上面的配置:

```
events {  
    worker_connections 1024;  
}
```

events 块涉及的指令主要影响 Nginx 服务器与用户的网络连接, 常用的设置包括是否开启对多 work process 下的网络连接进行序列化, 是否允许同时接收多个网络连接, 选取哪种事件驱动模型来处理连接请求, 每个 worker_process 可以同时支持的最大连接数等。

上述例子就表示每个 worker_process 支持的最大连接数为 1024. 这部分的配置对 Nginx 的性能影响较大, 在实际中应该灵活配置。

1.5.3、第三部分：http 块

```
1 http {  
2     include      mime.types;  
3     default_type  application/octet-stream;  
4  
5  
6     sendfile      on;  
7  
8     keepalive_timeout 65;  
9  
10    server {  
11        listen      80;  
12        server_name localhost;  
13  
14        location / {  
15            root      html;  
16            index  index.html index.htm;  
17        }  
18  
19        error_page   500 502 503 504  /50x.html;  
20        location = /50x.html {  
21            root      html;  
22        }  
23  
24    }  
25  
26 }
```

这算是 Nginx 服务器配置中最频繁的部分，代理、缓存和日志定义等绝大多数功能和第三方模块的配置都在这里。需要注意的是：http 块也可以包括 **http 全局块**、**server 块**。

①、http 全局块

http 全局块配置的指令包括文件引入、MIME-TYPE 定义、日志自定义、连接超时时间、单链接请求数上限等。

②、server 块

这块和虚拟主机有密切关系，虚拟主机从用户角度看，和一台独立的硬件主机是完全一样的，该技术的产生是为了节省互联网服务器硬件成本。

每个 http 块可以包括多个 server 块，而每个 server 块就相当于一个虚拟主机。而每个 server 块也分为全局 server 块，以及可以同时包含多个 location 块。

1、全局 server 块

最常见的配置是本虚拟机主机的监听配置和本虚拟主机的名称或 IP 配置。

2、location 块

一个 server 块可以配置多个 location 块。

这块的主要作用是基于 Nginx 服务器接收到的请求字符串（例如 server_name/uri-string），对虚拟主机名称（也可以是 IP 别名）之外的字符串（例如 前面的 /uri-string）进行匹配，对特定的请求进行处理。地址定向、数据缓存和应答控制等功能，还有许多第三方模块的配置也在这里进行。

1.6、Bug 解决

万一遇到这种问题，直接创建该目录及文件即可。

nginx: [emerg] open() "/var/run/nginx/nginx.pid" failed (2: No such file or directory)

解决方法：

(1) 进入 `cd /usr/local/nginx/conf/` 目录，编辑配置文件 `nginx.conf`

(2) 在配置文件中有个注释的地方：`#pid logs/nginx.pid;`

(3)

```
#user nobody;
worker_processes 4;
worker_cpu_affinity 0001 0010 0100 1000;
```

```
#error_log logs/error.log;
#error_log logs/error.log notice;
#error_log logs/error.log info;
```

```
pid logs/nginx.pid;
```

```
events {
    worker_connections 1024;
}
```

原来是注释，将注释放

(4) 在 `/usr/local/nginx` 目录下创建 `logs` 目录：`mkdir /usr/local/nginx/logs`

(5) 再次启动 nginx 服务：`cd /usr/local/nginx/sbin/` 问题解决

```
[root@centos7 sbin]# ./nginx
[root@centos7 sbin]# ps -ef | grep nginx
root      1155      1    0 21:37 ?        00:00:00 nginx: master process ./nginx
nobody    1156    1155    0 21:37 ?        00:00:00 nginx: worker process
nobody    1157    1155    0 21:37 ?        00:00:00 nginx: worker process
nobody    1158    1155    0 21:37 ?        00:00:00 nginx: worker process
nobody    1159    1155    0 21:37 ?        00:00:00 nginx: worker process
root      1161    1125    0 21:37 pts/0    00:00:00 grep --color=auto nginx
```

1.7、设置 nginx 开机自启

第一步：进入到/lib/systemd/system/目录

```
cd /lib/systemd/system/
```

第二步：创建 nginx.service 文件，并编辑

```
vim nginx.service
```

内容如下：

```
[Unit]
Description=nginx service
After=network.target

[Service]
Type=forking
ExecStart=/usr/local/nginx/sbin/nginx
ExecReload=/usr/local/nginx/sbin/nginx -s reload
ExecStop=/usr/local/nginx/sbin/nginx -s quit
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

[Unit]:服务的说明

Description:描述服务

After:描述服务类别

[Service]服务运行参数的设置

Type=forking 是后台运行的形式

ExecStart 为服务的具体运行命令

ExecReload 为重启命令

ExecStop 为停止命令

PrivateTmp=True 表示给服务分配独立的临时空间

注意：[Service]的启动、重启、停止命令全部要求使用绝对路径

[Install]运行级别下服务安装的相关设置，可设置为多用户，即系统运行级别为 3

保存退出。

第三步：加入开机自启动

```
systemctl enable nginx
```

如果不想开机自启动了，可以使用下面的命令取消开机自启动


```
systemctl disable nginx
```

第四步：服务的启动/停止/刷新配置文件/查看状态

```
# systemctl start nginx.service      启动nginx服务
# systemctl stop nginx.service       停止服务
# systemctl restart nginx.service     重新启动服务
# systemctl list-units --type=service 查看所有已启动的服务
# systemctl status nginx.service      查看服务当前状态
# systemctl enable nginx.service      设置开机自启动
# systemctl disable nginx.service     停止开机自启动
```

一个常见的错误

warning: nginx.service changed on disk. Run 'systemctl daemon-reload' to reload units.

直接按照提示执行命令 `systemctl daemon-reload` 即可。

```
systemctl daemon-reload .
```

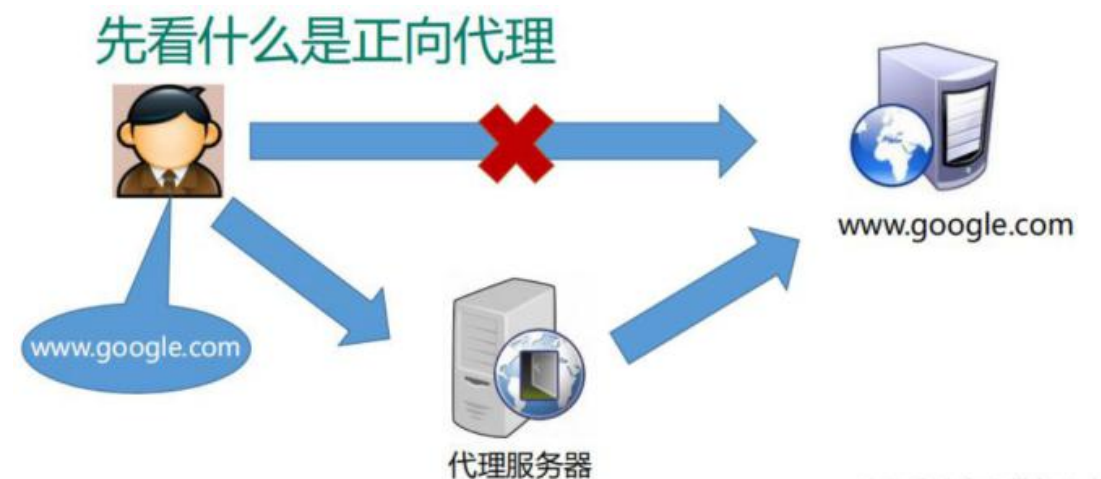
2、Nginx 反向代理

2.1、反向代理简介

2.1.1、正向代理

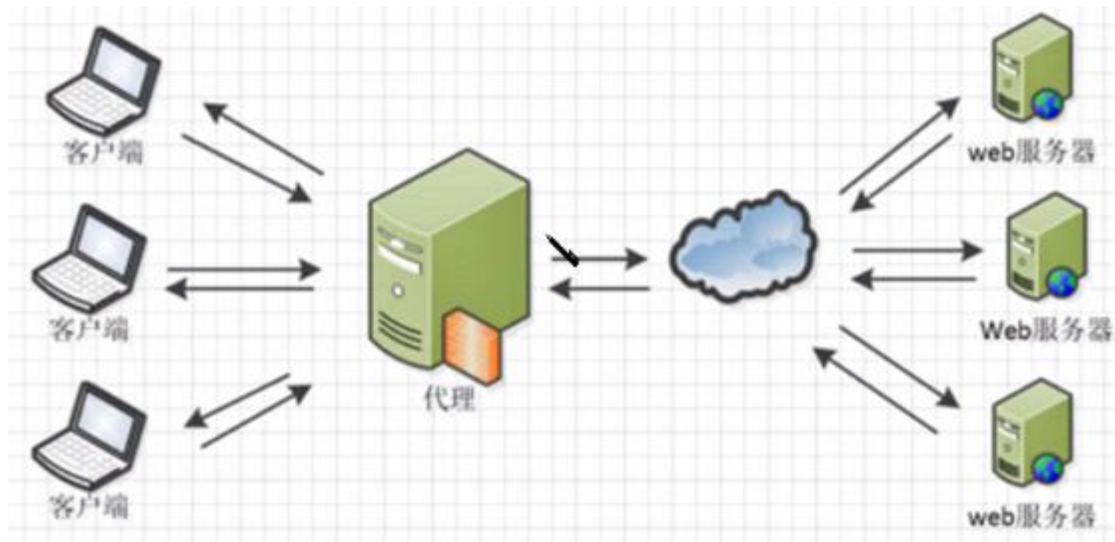
Nginx 不仅可以做反向代理，实现负载均衡。还能用作正向代理来进行上网等功能。

正向代理：如果把局域网外的 Internet 想象成一个巨大的资源库，则局域网中的客户端要访问 Internet，则需要通过代理服务器来访问，这种代理服务就称为正向代理。



正向代理 (vpn) 是为我们服务的，即为客户端服务的（正向代理的代理服务器是代理的客户端），客户端可以根据正向代理访问到它本身无法访问到的服务器资源。

正向代理 (vpn) 对我们是透明的，对服务端是非透明的，即服务端并不知道自己收到的是来自代理的访问还是来自真实客户端的访问。



2.1.2、反向代理

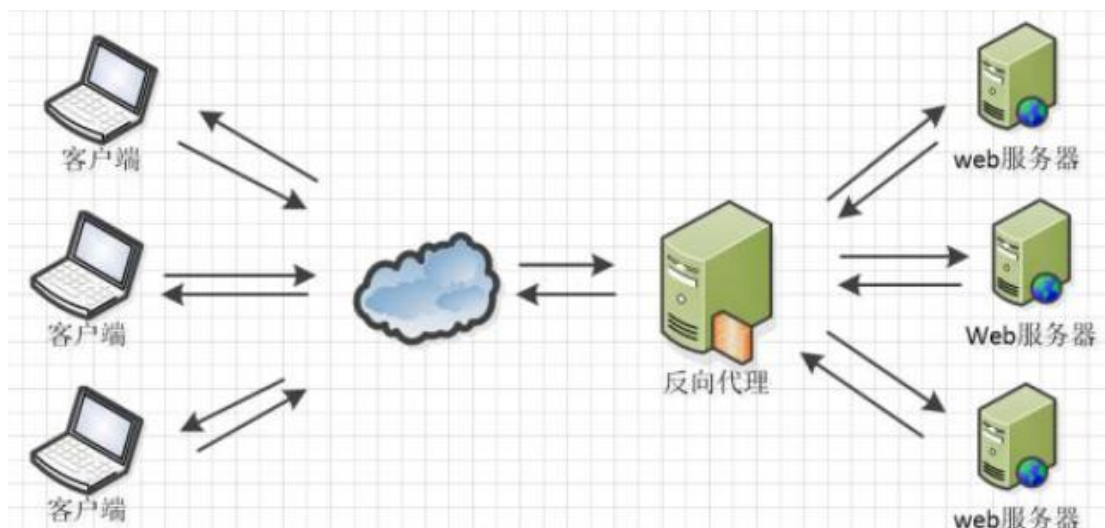
反向代理（Reverse Proxy）方式是指以代理服务器来接受 internet 上的连接请求，然后将请求转发给内部网络上的服务器，并将从服务器上得到的结果返回给 internet 上请求连接的客户端，此时代理服务器对外就表现为一个反向代理服务器。此时反向代理服务器和目标服务器对外就是一个服务器，暴露的是代理服务器地址，隐藏了真实服务器 IP 地址。

再说什么是反向代理



反向代理是为服务端服务的，反向代理可以帮助服务器接收来自客户端的请求，帮助服务器做请求转发，负载均衡等。

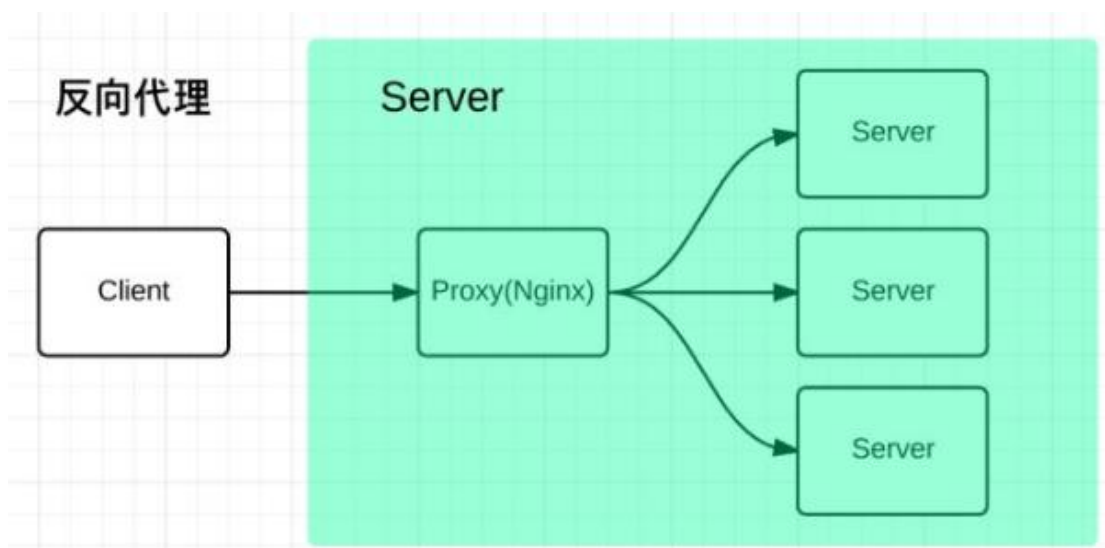
反向代理对服务端是透明的，对我们是非透明的，即我们并不知道自己访问的是代理服务器，而服务器知道反向代理在为他服务。



2.2、反向代理案例

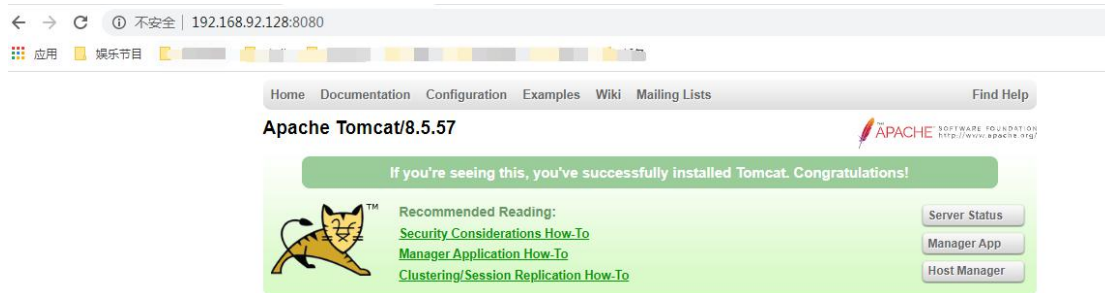
2.2.1、案例一

目标效果：

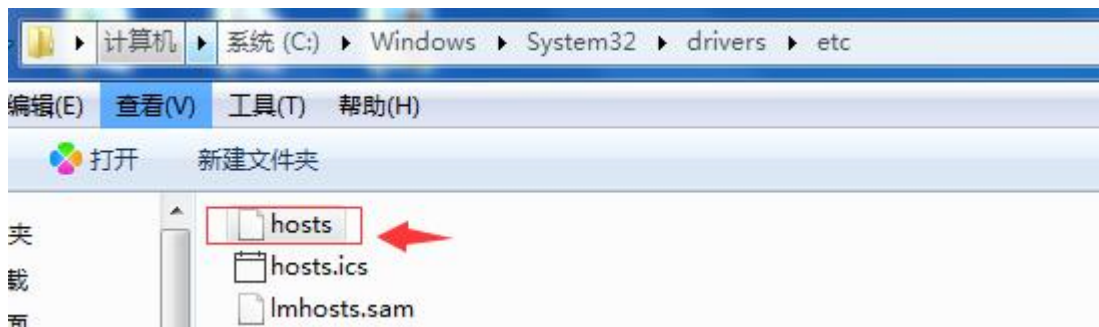


效果说明：服务端使用 nginx 反向代理，客户端访问 `www.123.com`，nginx 将请求转发到服务端的某个 tomcat 服务器上。

第一步：启动一个 `tomcat`，浏览器地址栏输入 `127.0.0.1:8080`，出现如下界面



第二步：通过修改本地 `host` 文件，将 `www.123.com` 映射到 `127.0.0.1`



配置如下：

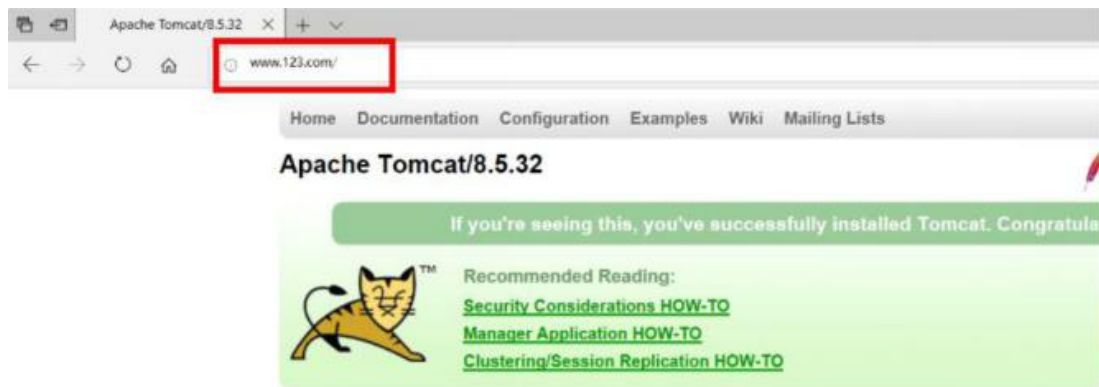
`192.168.92.128      www.123.com .`

配置完成之后，我们便可以通过 `www.123.com:8080` 访问到第一步出现的 Tomcat 初始界面。那么如何只需要输入 `www.123.com` 便可以跳转到 Tomcat 初始界面呢？便用到 nginx 的反向代理。

第三步：在 `nginx.conf` 配置文件中增加如下配置



如上配置：我们监听 80 端口，访问域名为 `www.123.com`，不加端口号时默认为 80 端口，故访问该域名时会跳转到 `127.0.0.1:8080` 路径上。重新加载 nginx 配置文件，然后在浏览器访问 `www.123.com`，结果如下：



2.2.2、案例二

目标效果:

使用 nginx 反向代理, 根据访问的路径跳转到不同端口的服务中 nginx 监听端口为 9001,

访问 `http://127.0.0.1:9001/edu/` 直接跳转到 `127.0.0.1:8081`

访问 `http://127.0.0.1:9001/vod/` 直接跳转到 `127.0.0.1:8082`

第一步:准备 tomcat:

准备两个 tomcat, 一个 8001 端口, 一个 8002 端口, 并准备好测试的页面

注意:

1. 修改第二个 tomcat 的三个端口号(8005、8080、8009)
2. 将/etc/profile 中的 tomcat 环境变量配置去掉, 并刷新该文件、重启虚拟机
3. 在防火墙中放开新 tomcat 的访问端口号

第二步:修改 nginx 的配置文件, 在 http 块中添加 server {}

```
server {  
    listen 9001;  
    server_name localhost;  
  
    location ~ /edu/ {  
        proxy_pass http://127.0.0.1:8080;  
        index index.html index.htm;  
    }  
  
    location ~ /vod/ {  
        proxy_pass http://127.0.0.1:8081;  
        index index.html index.htm;  
    }  
}
```

```

server {
    listen      9001;
    server_name localhost;

    location ~ /edu/ {
        proxy_pass http://127.0.0.1:8080;
        index index.html index.htm;
    }

    location ~ /vod/ {
        proxy_pass http://127.0.0.1:8081;
        index index.html index.htm;
    }
}

```

注意：

1. 重启 nginx
2. 要记得放开防火墙的 9001 端口号哦。

第三步：测试 nginx

先在端口号为 8080 的 tomcat 的 webapps 目录里创建一个目录为 **edu** 的目录, 在里面创建 a.html, 写明 **8080**
 再在端口号为 8081 的 tomcat 的 webapps 目录里创建一个目录为 **vod** 的目录, 在里面创建 a.html, 写明 **8081**
 在浏览器访问：

<http://192.168.92.128/edu/a.html>

<http://192.168.92.128/vod/a.html>

效果如下所示：



2.2.3、Location 指令说明

该指令用于匹配 URL。语法如下：

语法如下：**location [=|~|~*|^~|@] pattern{.....}**

1、=：用于**不含**正则表达式的 uri 前，要求请求字符串与 uri **严格匹配**，如果匹配成功，就停止继续向下搜索并立即处理该请求。

```
server {  
    server_name localhost  
        location = /abc {  
            .....  
        }  
}
```

匹配：

<http://192.168.200.168/abc>

不匹配：

<http://192.168.200.168/abc/>

<http://192.168.200.168/abcef>

2、没有修饰符，必须以指定模式开始[**模糊匹配**]

```
server {  
    server_name localhost;  
    location /a/b { #匹配路径以/a/b开头的路径，区分大小写  
        .....  
    }  
}
```

<http://192.168.200.168/a/b>

<http://192.168.200.168/a/b/asd>

<http://192.168.200.168/a/b/c/sd?id=1>

3、~：用于表示 uri **包含**正则表达式，并且**区分大小写**

```
server {  
    server_name localhost;  
    location ~ ^/abc$ {  
        .....  
    }  
}
```

匹配：

<http://192.168.200.168/abc>

<http://192.168.200.168/abc?k=v>

不匹配：

<http://192.168.200.168/AbC>

<http://192.168.200.168/abc/>

<http://192.168.200.168/abcde>

4、~*：用于表示 uri **包含**正则表达式，并且**不区分大小写**。


```
server {
server_name localhost;
location ~* ^/abc$ {
    .....
}
}
```

匹配：

<http://192.168.200.168/abc>

<http://192.168.200.168/ABC>

不匹配：

<http://192.168.200.168/abc/>

<http://192.168.200.168/abcde>

5、`^~`：用于不含正则表达式的 uri 前，要求 Nginx 服务器找到标识 uri 和请求字符串匹配度最高的 location 后，立即使用此 location 处理请求，而不再使用 location 块中的正则 uri 和请求字符串做匹配。

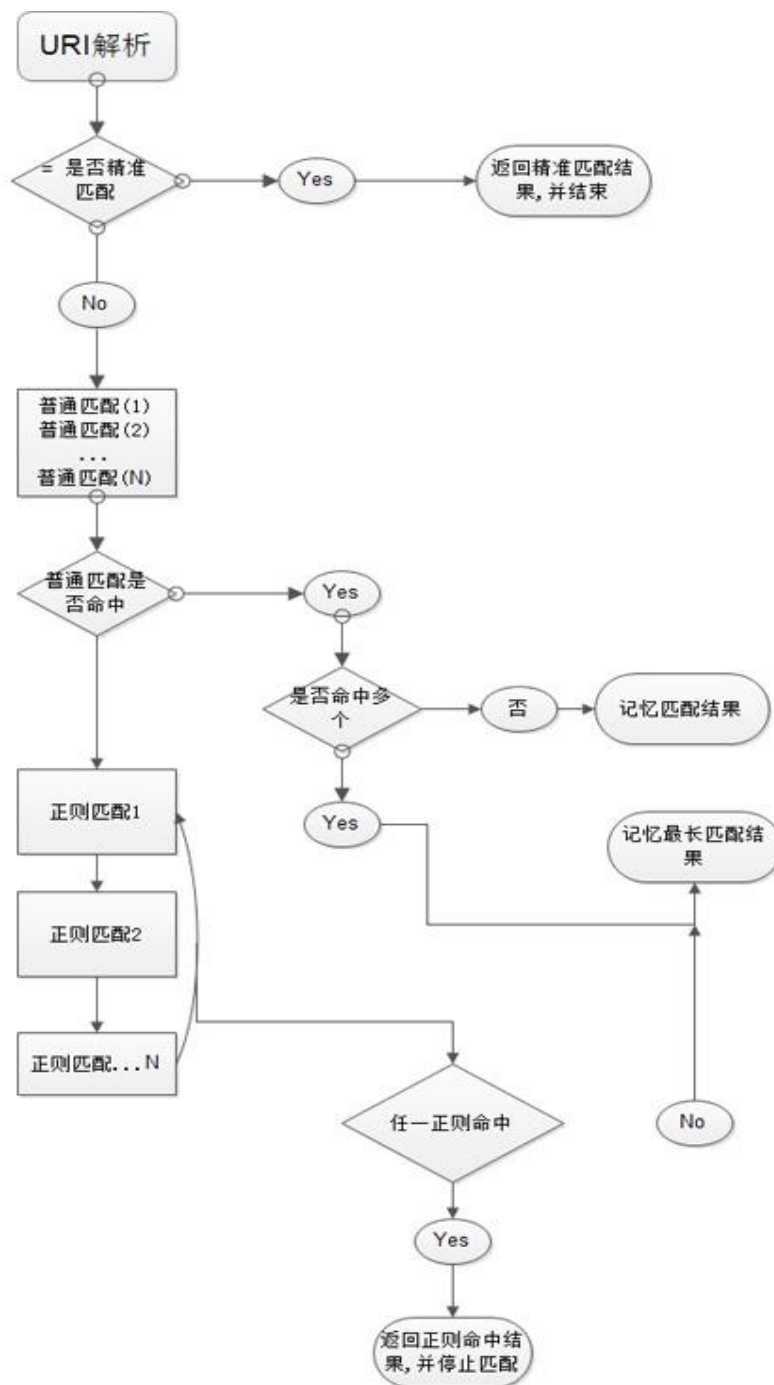
注意：如果 uri 包含正则表达式，则必须要有 `~` 或者 `~*` 标识。

2.2.4、优先级说明

查找顺序和优先级

<code>location = /uri</code>	=开头表示精确匹配，只有完全匹配上才能生效，匹配成功后 立即停止其他匹配 。
<code>location ^~ /uri</code>	<code>^~</code> 开头对URL路径进行 前缀匹配 ，并且在正则之前，匹配成功后 立即停止其他匹配 。
<code>location ~ pattern</code>	<code>~</code> 开头表示区分大小写的 正则匹配 。
<code>location ~* pattern</code>	<code>~*</code> 开头表示不区分大小写的 正则匹配 。
<code>location /uri</code>	不带任何修饰符也表示前缀匹配，但在正则匹配之后，而且nginx会根据配置的 长短 来进行匹配，记忆最长匹配，但不会停止匹配。
<code>location /</code>	通用匹配，任何未匹配到其它location的请求都会匹配到，相当于switch中的default。

注意：上述只有正则表达式的匹配和配置顺序有关，普通字符串和配置顺序无关。



总结: location的命中过程是这样的

- 1: 先判断精准命中, 如果命中, 立即返回结果并结束解析过程
- 2: 判断普通命中, 如果有多个命中, “记录”下来“最长”的命中结果 (注意: 记录但不结束, 最长的为准)
- 3: 继续判断正则表达式的解析结果, 按配置里的正则表达式顺序为准, 由上到下开始匹配, 一旦匹配成功1个, 立即返回结果, 并结束解析过程.

延伸分析: a: 普通命中 顺序无所谓, 是因为按命中的长短来确定的.
b: 正则命中, 顺序有所谓, 因为是从前往后命中的

3、Nginx 负载均衡

3.1、什么是负载均衡

客户端发送多个请求到服务器，服务器处理请求，有一些可能要与数据库进行交互，服务器处理完毕后，再将结果返回给客户端。

这种架构模式对于早期的系统相对单一，并发请求相对较少的情况下是比较适合的，成本也低。但是随着信息数量的不断增长，访问量和数据量的飞速增长，以及系统业务的复杂度增加，这种架构会造成服务器相应客户端的请求日益缓慢，并发量特别大的时候，还容易造成服务器直接崩溃。很明显这是由于服务器性能的瓶颈造成的问题，那么如何解决这种情况呢？

我们首先想到的可能是升级服务器的配置，比如提高 CPU 执行频率，加大内存等提高机器的物理性能来解决此问题，但是我们知道摩尔定律的日益失效，硬件的性能提升已经不能满足日益提升的需求了。最明显的一个例子，天猫双十一当天，某个热销商品的瞬时访问量是极其庞大的，那么类似上面的系统架构，将机器都增加到现有的顶级物理配置，都是不能够满足需求的。那么怎么办呢？

上面的分析我们去掉了增加服务器物理配置来解决问题的办法，也就是说纵向解决问题的办法行不通了，那么横向增加服务器的数量呢？这时候集群的概念产生了，单个服务器解决不了，我们增加服务器的数量，然后将请求分发到各个服务器上，将原先请求集中到单个服务器上的情况改为将请求分发到多个服务器上，将负载分发到不同的服务器，也就是我们所说的**负载均衡**。



3.2、配置演示

3.2.1、目标效果:如上图所示

3.2.2、具体配置

第一步:准备两个 Tomcat，并在每个 tomcat 下的 ROOT 目录下分别添加有区分的 a.html

第二步:在 nginx.conf 中进行配置

```
upstream myserver {  
    server 192.168.92.128:8080;  
    server 192.168.92.128:8081;  
}  
  
server {  
    listen      80;  
    server_name www.123.com;  
    #charset koi8-r;  
    #access_log logs/host.access.log main;  
    location / {  
        proxy_pass http://myserver;  
        proxy_connect_timeout 10;  
    }  
    error_page 500 502 503 504 /50x.html;  
    location = /50x.html {  
        root html;  
    }  
}
```

proxy_connect_timeout :后端服务器连接的超时时间_发起握手等候响应超时时间

upstream 称为上游服务器，即真实处理请求的业务服务器。

第三步: 在 windows 浏览器多次访问: <http://192.168.92.128/a.html> 会发现区别

3.3、几种负载均衡策略

随着互联网信息的爆炸性增长，负载均衡（load balance）已经不再是一个很陌生的话题，顾名思义，负载均衡即是将负载分摊到不同的服务单元，既保证服务的可用性，又保证响应足够快，给用户很好的体验。快速增长的访问量和数据流量催生了各式各样的负载均衡产品，很多专业的负载均衡硬件提供了很好的功能，但却价格不菲，这使得负载均衡软件大受欢迎，nginx 就是其中的一个，在 linux 下有 Nginx、LVS、Haproxy 等服务可以提供负载均衡服务，而且 Nginx 提供了几种分配方式(策略)：

1、轮询（默认）

每个请求按时间顺序逐一分配到不同的后端服务器，如果后端服务器 down 掉，能自动剔除。

2、weight

weight 代表权,重默认为 1,权重越高被分配的客户端越多

指定轮询几率, weight 和访问比率成正比,用于后端服务器性能不均的情况。 例如:

```
upstream myserver {  
    server 192.168.92.128:8080 weight=10;  
    server 192.168.92.128:8081 weight=5;  
}
```



3、ip_hash

每个请求按访问 ip 的 hash 结果分配,这样每个访客固定访问一个后端服务器,可以解决 session 的问题。例如:

```
upstream myserver {  
    ip_hash;  
    server 192.168.92.128:8080;  
    server 192.168.92.128:8081;  
}
```



4、fair (第三方)

按后端服务器的响应时间来分配请求,响应时间短的优先分配。

```
upstream myserver {  
    server 192.168.92.128:8080;  
    server 192.168.92.128:8081;  
    fair;  
}
```



5、其它参数

注:

```
upstream myServer {  
    server 139.199.203.169:8080 down;  
    server 139.199.203.169:8081 weight=2;  
    server 139.199.203.169:8082 backup;  
}
```

1) down

表示单前的server暂时不参与负载

2) Weight

默认为1.weight越大,负载的权重就越大。

3) Backup

其它所有的非backup机器down或者忙的时候,请求backup机器。所以这台机器压力会最轻。

4、Nginx 动静分离

由于 nginx 擅长处理静态资源,而 tomcat 擅长处理动态资源,所以为了加快网站的解析速度,可以把动态页面和静态页面由不同的服务器来解析,加快解析速度。降低原来单个服务器的压力。



第一步：在两个 tomcat 的 webapps 目录下的 a.html 中添加 img 图片引用

在 8080 端口号的 tomcat 的 webapps 目录下的 ROOT 目录中的 index.jsp 中添加：

```
<h1>主tomcat:8080</h1>
</img>
```

在 8081 端口号的 tomcat 的 webapps 目录下的 ROOT 目录中的 index.jsp 中添加：

```
<h1>主tomcat:8081</h1>
</img>
```

第二步：在 nginx 的配置文件中配置：

```
location / {
    proxy_pass http://myserver;
    index index.html index.htm;
}
```

```
location ~ \.(gif|jpg|jpeg|png|bmp|swf)$ {
    root /usr/share/nginx/html;
}
```

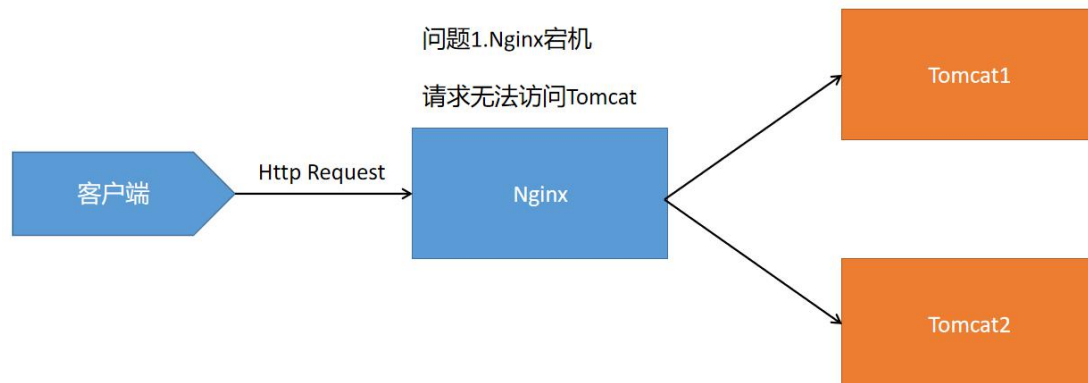
```
location ~ \.(jsp|do|action)$ {
    proxy_pass http://myserver;
}
```

第三步：在 /usr/share/nginx/html 目录中存放的 1.jpg 图片

第四步：在浏览器访问：<http://www.123.com/>

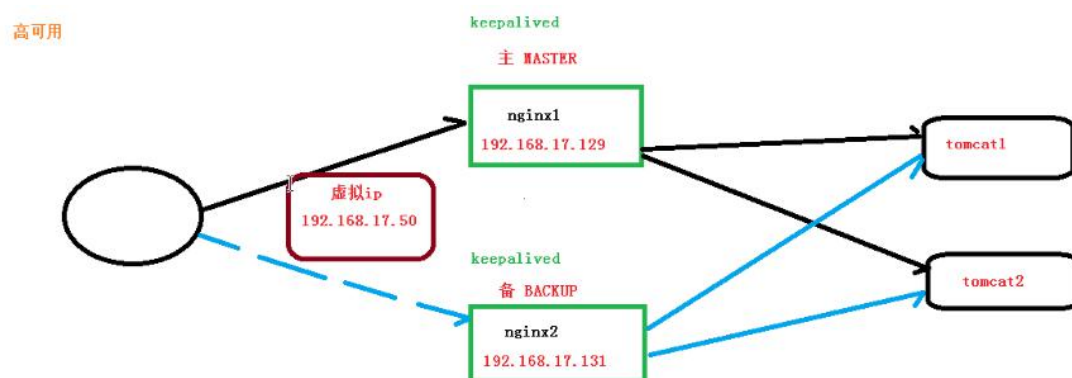


5、Nginx 集群



在上图中，nginx 都是作为单台服务器单独运行的，如果 nginx 本身挂掉，那他后面的 tomcat 中的项目我们就访问不了了，为了解决 nginx 单节点问题，我们这里借助于 nginx 集群实现 Nginx 高可用。Nginx 集群有两种模式：一种是主从模式 一种是双主模式。

5.1、第一种：keepalived+Nginx 高可用集群(主从模式)



5.1.1、高可用准备工作

- 第一步：需要两台服务器并且安装好 nginx
- 第二步：在两台 nginx 所在的虚拟机上安装 keepalived
- 第三步：完成高可用配置(主从配置)

5.1.2、再准备另一台服务器并安装好 nginx

- 第一步：克隆：
- 第二步：修改网络：

```
vi /etc/sysconfig/network-scripts/ifcfg-ens33
```

去掉 UUID
修改 ip:介于 3-254 之间，并且不能和其它虚拟机 ip 冲突

第三步:修改主机名

```
vi /etc/hostname
```

第四步:编辑主机名和 ip 的映射

```
vi /etc/hosts
```

第五步:重启 reboot

第六步:检测

```
ping www.baidu.com
```

```
ip a
```

5.1.3、在两台 nginx 所在的虚拟机上安装 keepalived

(1)使用 yum 命令进行安装

```
yum install keepalived -y
```

(2)安装之后,在 etc 里面生成目录 keepalived,有文件 keepalived.conf

5.1.4、完成高可用配置(主从都修改配置)

(1)修改/etc/keepalived/keepalived.conf 配置文件

```
global_defs {
    notification_email {
        acassen@firewall.loc
        failover@firewall.loc
        sysadmin@firewall.loc
    }
    notification_email_from Alexandre.Cassen@firewall.loc
    smtp_server 192.168.17.129
    smtp_connect_timeout 30
    router_id LVS_DEVEL
}

vrrp_script chk_http_port {
    script "/usr/local/src/nginx_check.sh"
    interval 2 # (检测脚本执行的间隔)
    weight 2
}

vrrp_instance VI_1 {
    state MASTER # 备份服务器上将 MASTER 改为 BACKUP
    interface ens33 //网卡
    virtual_router_id 51 # 主、备机的 virtual_router_id 必须相同
    priority 90 # 主、备机取不同的优先级,主机值较大,备份机值较小
    advert_int 1
    authentication {
        auth_type PASS
        auth_pass 1111
    }
}
```



```
virtual_ipaddress {
    192.168.92.50 // VRRP H 虚拟地址
}
}
```

(2)在/usr/local/src 中添加检测脚本

```
#!/bin/bash
A=`ps -C nginx --no-header |wc -l`
if [ $A -eq 0 ];then
    /usr/local/nginx/sbin/nginx
    sleep 2
    if [ `ps -C nginx --no-header |wc -l` -eq 0 ];then
        killall keepalived
    fi
fi
```

(3)把两台服务器上 nginx 和 和 keepalived

先启动两台服务器上的 tomcat (每个服务器有两个 tomcat,共 4 个 tomcat 都启动)

启动 nginx: ./nginx

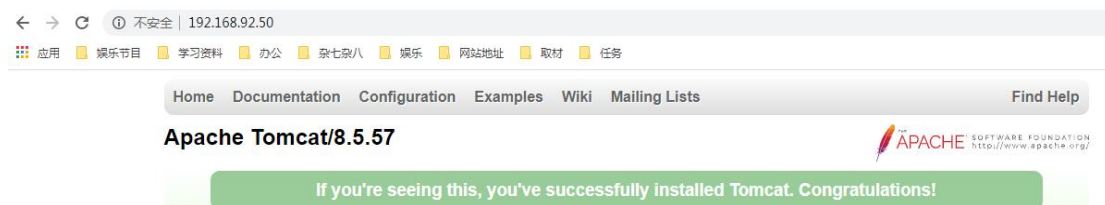
启动 keepalived: systemctl start keepalived.service

5.1.5、最终测试 1

(1)查看虚拟 ip 是否绑定成功

```
[root@centos73 bin]# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN qlen 1
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP qlen 1000
    link/ether 00:0c:29:e1:31:3d brd ff:ff:ff:ff:ff:ff
    inet 192.168.92.148/24 brd 192.168.92.255 scope global ens33
        valid_lft forever preferred_lft forever
    inet 192.168.92.50/32 scope global ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::b242:c069:44c1:e40b/64 scope link
        valid_lft forever preferred_lft forever
```

(2)在浏览器地址栏输入虚拟 ip 地址 192.168.17.50,在浏览器可以看到 tomcat 网页,说明虚拟 ip 可用



5.1.6、继续测试 2

把主服务器(192.168.92.128)的 nginx 和 keepalived 停掉,再在浏览器输入虚拟 ip, 应该也可以访问

systemctl stop keepalived #先停掉 keepalived

./nginx -s stop #再关闭 nginx

再次访问虚拟 ip, 发现还是可以访问的。

```

<  →  ①  不安全  192.168.92.50
应用  娱乐节目  学习资料  办公  杂七杂八  娱乐  网站地址  取材  任务

Home  Documentation  Configuration  Examples  Wiki  Mailing Lists  Find Help

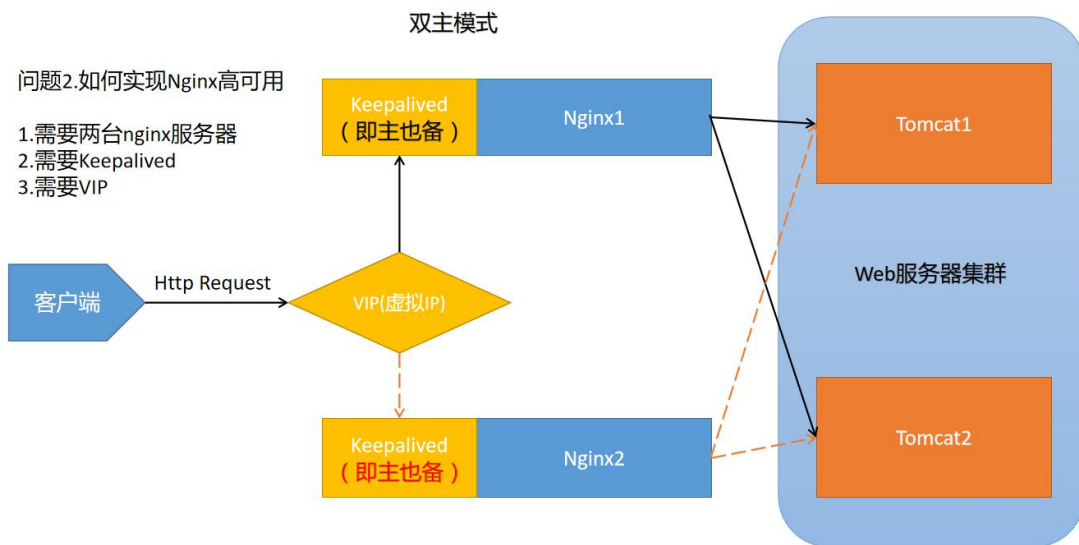
Apache Tomcat/8.5.57    SOFTWARE FOUNDATION
http://www.apache.org/

If you're seeing this, you've successfully installed Tomcat. Congratulations!

[root@centos73 bin]# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN qlen 1
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP qlen 1000
    link/ether 00:0c:29:e1:31:3d brd ff:ff:ff:ff:ff:ff
    inet 192.168.92.148/24 brd 192.168.92.255 scope global ens33
        valid_lft forever preferred_lft forever
    inet 192.168.92.50/32 scope global ens33 → 虚拟ip已经切换到备机
        valid_lft forever preferred_lft forever
    inet6 fe80::b242:c069:44c1:e40b/64 scope link
        valid_lft forever preferred_lft forever

```

5.2、第一种：keepalived+Nginx 高可用集群(双主模式)



5.2.1、主机配置

在 keepalived.conf 最后一行添加

```

vrrp_instance VI_2 {
    state BACKUP
    interface ens33
    virtual_router_id 52 #新主备的路由 id:52
    priority 70          #和另一个从机中的优先级一致即可。
    advert_int 1

```

```

authentication {
    auth_type PASS
    auth_pass 1111
}
virtual_ipaddress {
    192.168.92.51 #新的虚拟 ip
}
}

```

5.2.2、备机配置

```

vrrp_instance VI_2 {
    state MASTER
    interface ens33
    virtual_router_id 52
    priority 90
    advert_int 1
    authentication {
        auth_type PASS
        auth_pass 1111
    }
    virtual_ipaddress {
        192.168.92.51
    }
}

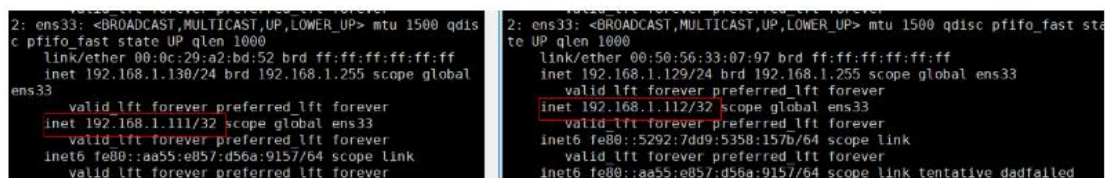
```

5.2.3、重启主备的 keepalived

重启主备的 keepalived: `systemctl restart keepalived`

执行: `ip addr`

主备各有一个 VIP



```

2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP qlen 1000
    link/ether 00:0c:29:a2:bd:52 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.130/24 brd 192.168.1.255 scope global ens33
        valid_lft forever preferred_lft forever
    inet 192.168.1.111/32 scope global ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::aa55:e857:d56a:9157/64 scope link
        valid_lft forever preferred_lft forever

```

```

2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP qlen 1000
    link/ether 00:50:56:33:07:97 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.129/24 brd 192.168.1.255 scope global ens33
        valid_lft forever preferred_lft forever
    inet 192.168.1.112/32 scope global ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::5292:7dd9:5358:157b/64 scope link
        valid_lft forever preferred_lft forever
    inet6 fe80::aa55:e857:d56a:9157/64 scope link tentative dadfailed

```

5.2.4、编辑主备服务器上的两个 tomcat(共 4 个 tomcat)

在主服务器 tomcat 上的端口号为 8080 的 index.jsp 设置显示: 主 tomcat8080

在主服务器 tomcat 上的端口号为 8081 的 index.jsp 设置显示: 主 tomcat8081

特别说明: 在两台服务器的 nginx 的配置文件中只访问了主服务器上的两个 tomcat, 所以这里不需要配置从服务器上的 tomcat。

5.2.5、测试双主模式

启动主备的 keepalived 和 nginx

浏览器访问：<http://192.168.92.50/>



访问另一个虚拟 ip:<http://192.168.92.51/> 发现也是可以使用的



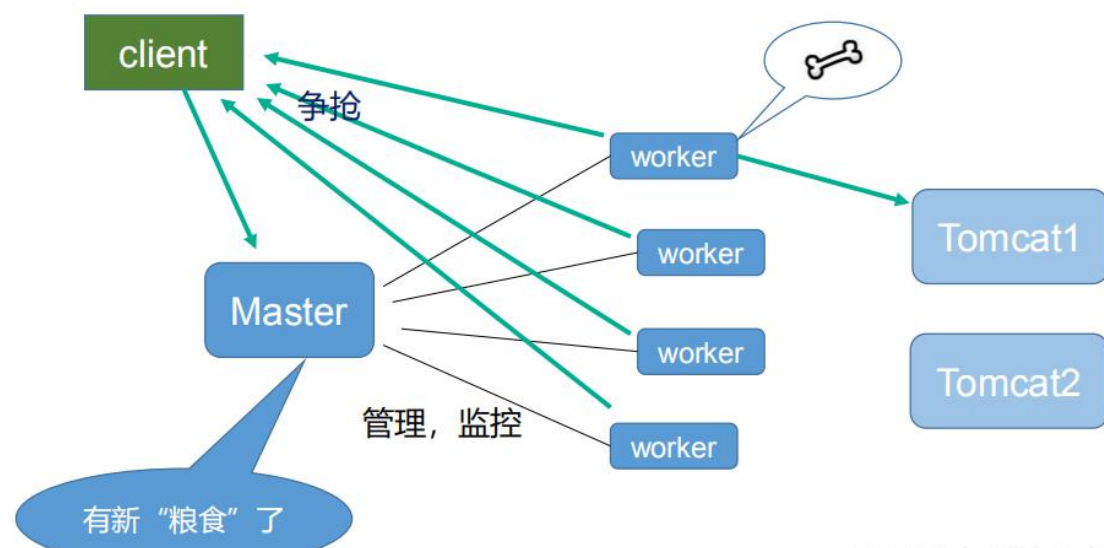
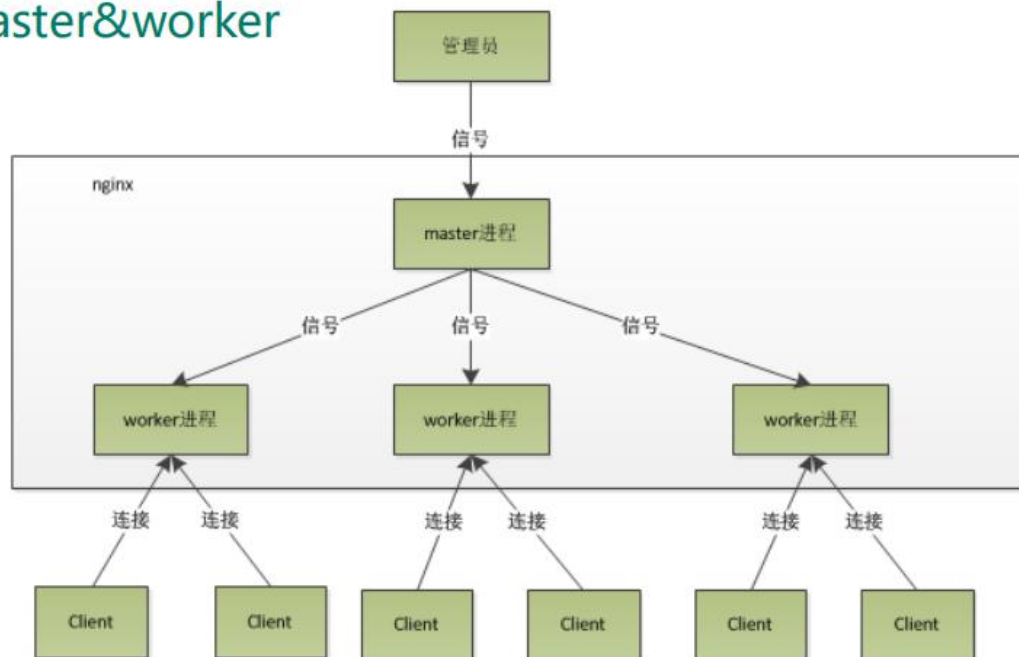
再测：将主服务器的 keepalived 关掉。再访问第二个虚拟 ip：<http://192.168.92.51/>



发现依旧可以使用。

6、Nginx 原理及优化配置

master&worker



master-workers 的机制的好处

首先，对于每个 worker 进程来说，独立的进程，不需要加锁，所以省掉了锁带来的开销，同时在编程以及问题查找时，也会方便很多。**其次**，采用独立的进程，可以让互相之间不会互相影响，一个进程退出后，其它进程还在工作，服务不会中断，master 进程则很快启动新的 worker 进程。**当然**，worker 进程的异常退出，肯定是程序有 bug 了，异常退出，会导致当前 worker 上的所有请求失败，不过不会影响到所有请求，所以降低了风险。

需要设置多少个 worker

Nginx 同 redis 类似都采用了 **io 多路复用机制**，每个 worker 都是一个独立的进程，但每个进程里只有一个主线程，通过**异步非阻塞**的方式来处理请求，即使是千上万个请求也不在话下。每个 worker 的线程可以把一个

cpu 的性能发挥到极致。所以 **worker 数和服务器的 (cpu 个数*核数) 相等是最为适宜**的。设少了会浪费 cpu , 设多了会造成 cpu 频繁切换上下文带来的损耗。

#设置 worker 数量

worker_processes 4

#work 绑定 cpu 个数*核数(4 work 绑定 4cpu 核数)。

worker_cpu_affinity 0001 0010 0100 1000

#work 绑定 cpu (4 work 绑定 8cpu 中的 4 个) 。

worker_cpu_affinity 00000001 00000010 00000100 00001000 00010000 00100000 01000000 10000000

#连接数

worker_connections 1024

这个值是表示每个 worker 进程所能建立连接的最大值，所以，一个 nginx 能建立的最大连接数，应该是 worker_connections * worker_processes。当然，这里说的是最大连接数，对于 HTTP 请求本地资源来说，能够支持的**最大并发数量**是 worker_connections * worker_processes ,如果是支持 http1.1 的浏览器每次访问要占两个连接 所以普通的静态访问最大并发数是：**worker_connections * worker_processes /2** ,而如果是 HTTP 作为反向代理来说，最大并发数量应该是 **worker_connections * worker_processes/4**。

因为作为反向代理服务器，每个并发会建立与客户端的连接和与后端服务的连接，会占用两个连接。

了解：每个连接指的是对系统中的文件进行读或写操作，进程最大连接数指的是进程最大可打开文件数，受限与操作系统，可以通过 ulimit -n 命令查询(默认 1024),也可以通过 ulimit -SHn 65535 修改进程最大可打开文件数(数字不是越大越好，nginx 占用内存越小处理性能越高)

配置文件:nginx.conf

#安全问题，建议用 nobody,不要用 root.

#user nobody;

#worker 数和服务器的 cpu 数相等是最为适宜

worker_processes 2;

#work 绑定 cpu(4 work 绑定 4cpu)

worker_cpu_affinity 0001 0010 0100 1000

#work 绑定 cpu (4 work 绑定 8cpu 中的 4 个) 。

worker_cpu_affinity 0000001 00000010 00000100 00001000

#error_log path(存放路径) level(日志等级) path 表示日志路径，level 表示日志等级，

#具体如下：[**debug** | info | notice | **warn** | **error** | crit]

#从左至右，日志详细程度逐级递减，即 debug 最详细，crit 最少，默认为 **crit**。

#error_log logs/error.log;

#error_log logs/error.log notice;

#error_log logs/error.log info;

```
#pid logs/nginx.pid;
```

```
events {
```

#这个值是表示每个 worker 进程所能建立连接的最大值，所以，一个 nginx 能建立的最大连接数，应该是 `worker_connections * worker_processes`。

#当然，这里说的是最大连接数，对于 HTTP 请求本地资源来说，能够支持的最大并发数量是 `worker_connections * worker_processes`，

#如果是支持 http1.1 的浏览器每次访问要占两个连接，

#所以普通的静态访问最大并发数是：`worker_connections * worker_processes / 2`，

#而如果是 HTTP 作为反向代理来说，最大并发数量应该是 `worker_connections * worker_processes / 4`。

#因为作为反向代理服务器，每个并发会建立与客户端的连接和与后端服务的连接，会占用两个连接。

```
worker_connections 1024;
```

#这个值是表示 nginx 要支持哪种多路 io 复用。

#一般的 Linux 选择 `epoll`，如果是(*BSD)系列的 Linux 使用 `kqueue`。

#windows 版本的 nginx 不支持多路 IO 复用，这个值不用配。

```
use epoll;
```

当一个 worker 抢占到一个链接时，是否尽可能的让其获得更多的连接,默认是 off。

```
multi_accept on; //并发量大时缓解客户端等待时间。
```

默认是 on,开启 nginx 的抢占锁机制。

```
accept_mutex on; //master 指派 worker 抢占锁
```

```
}
```

```
http {
```

#当 web 服务器收到静态的资源文件请求时，依据请求文件的后缀名在服务器的 MIME 配置文件中找到对应的 MIME Type，再根据 MIME Type 设置 HTTP Response 的 Content-Type，然后浏览器根据 Content-Type 的值处理文件。

```
include mime.types; #/usr/local/nginx/conf/mime.types
```

#如果 不能从 mime.types 找到映射的话，用以下作为默认值-二进制

```
default_type application/octet-stream;
```

#日志位置

```
access_log logs/host.access.log main;
```

#一条典型的 accesslog：

#101.226.166.254 - - [21/Oct/2013:20:34:28 +0800] "GET /movie_cat.php?year=2013 HTTP/1.1" 200 5209 "http://www.baidu.com" "Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1; Trident/4.0; SLCC2; .NET CLR 2.0.50727; .NET CLR 3.5.30729; .NET CLR 3.0.30729; Media Center PC 6.0; MDDR; .NET4.0C; .NET4.0E; .NET CLR 1.1.4322; Tablet PC 2.0); 360Spider"

#1) 101.226.166.254:(用户 IP)

#2) [21/Oct/2013:20:34:28 +0800] : (访问时间)

#3) GET : http 请求方式, 有 GET 和 POST 两种

#4) /movie_cat.php?year=2013 : 当前访问的网页是动态网页, movie_cat.php 即请求的后台接口, year=2013 为具体接口的参数

#5) 200 : 服务状态, 200 表示正常, 常见的还有, 301 永久重定向、4XX 表示请求出错、5XX 服务器内部错误

#6) 5209 : 传送字节数为 5209, 单位为 byte

#7) "http://www.baidu.com" : refer:即当前页面的上一个网页

#8) "Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1; Trident/4.0; SLCC2; .NET CLR 2.0.50727; .NET CLR 3.5.30729; .NET CLR 3.0.30729; Media Center PC 6.0; MDDR; .NET4.0C; .NET4.0E; .NET CLR 1.1.4322; Tablet PC 2.0); 360Spider" : agent 字段: 通常用来记录操作系统、浏览器版本、浏览器内核等信息

```
log_format main '$remote_addr - $remote_user [$time_local] "$request" '
                '$status $body_bytes_sent "$http_referer" '
                '"$http_user_agent" "$http_x_forwarded_for";
```

#开启从磁盘直接到网络的文件传输, 适用于有大文件上传下载的情况, 提高 IO 效率。

```
sendfile on; //大文件传递优化, 提高效率
```

#一个请求完成之后还要保持连接多久, 默认为 0, 表示完成请求后直接关闭连接。

```
#keepalive_timeout 0;
```

```
keepalive_timeout 65;
```

#开启或者关闭 gzip 模块

```
#gzip on; //文件压缩, 再传输, 提高效率
```

#设置允许压缩的页面最小字节数, 页面字节数从 header 头中的 Content-Length 中进行获取。

```
#gzip_min_lenth 1k; //超过该大小开始压缩, 否则不用压缩
```

gzip 压缩比, 1 压缩比最小处理速度最快, 9 压缩比最大但处理最慢 (传输快但比较消耗 cpu)

```
#gzip_comp_level 4;
```

#匹配 MIME 类型进行压缩, (无论是否指定) "text/html" 类型总是会被压缩的。

```
#gzip_types types text/plain text/css application/json application/x-javascript text/xml
```

#动静分离

#服务器端静态资源缓存, 最大缓存到内存中的文件, 不活跃期限

```
open_file_cache max=655350 inactive=20s;
```

#活跃期限内最少使用的次数，否则视为不活跃。

```
open_file_cache_min_uses 2;
```

#验证缓存是否活跃的时间间隔

```
open_file_cache_valid 30s;
```

```
upstream myserver{
```

1、轮询（默认）

每个请求按时间顺序逐一分配到不同的后端服务器，如果后端服务器 down 掉，能自动剔除。

2、指定权重

指定轮询几率，weight 和访问比率成正比，用于后端服务器性能不均的情况。

#3、IP 绑定 ip_hash

每个请求按访问 ip 的 hash 结果分配，这样每个访客固定访问一个后端服务器，可以解决 session 的问题。

#4、备机方式 backup

正常情况下不访问设定为 backup 的备机，只有当所有非备机全都宕机的情况下，服务才会进备机。
当非备机启动后，自动切换到非备机

ip_hash;

```
server 192.168.161.132:8080 weight=1;
```

```
server 192.168.161.132:8081 weight=1 backup;
```

#5、fair（第三方）公平，需要安装插件才能用

#按后端服务器的响应时间来分配请求，响应时间短的优先分配。

#6、url_hash（第三方）

#按访问 url 的 hash 结果来分配请求，使每个 url 定向到同一个后端服务器，后端服务器为缓存时比较有效。

```
# ip_hash;
```

```
server 192.168.161.132:8080 weight=1;
```

```
server 192.168.161.132:8081 weight=1;
```

```
#fair
```

```
#hash $request_uri
```

```
#hash_method crc32
```

```
}
```

```
server {
```

#监听端口号

```
listen 80;
```

#服务名

```
server_name 192.168.161.130;
```

```

#字符集
#charset utf-8;

#location [=|~|~*|^~] /uri/ { ... }
# = 精确匹配
# ~ 正则匹配，区分大小写
# ~* 正则匹配，不区分大小写
# ^~ 关闭正则匹配
#匹配原则：
# 1、所有匹配分两个阶段，第一个叫普通匹配，第二个叫正则匹配。
# 2、普通匹配，首先通过 "=" 来匹配完全精确的 location
#    2.1、如果没有精确匹配到，那么按照最大前缀匹配的原则，来匹配 location
#    2.2、如果匹配到的 location 有 ^~,则以此 location 为匹配最终结果，如果没有那么会把匹
配的结果暂存，继续进行正则匹配。
# 3、正则匹配，依次从上到下匹配前缀是~或~*的 location，一旦匹配成功一次，则立刻以此
location 为准，不再向下继续进行正则匹配。
# 4、如果正则匹配都不成功，则继续使用之前暂存的普通匹配成功的 location.
#不是以波浪线开头的都是普通匹配。

location / { # 匹配任何查询，因为所有请求都以 / 开头。但是正则表达式规则和长的块规则
将被优先和查询匹配。

#定义服务器的默认网站根目录位置
    root    html;//相对路径，省略了./          /user/local/nginx/html 路径

#默认访问首页索引文件的名称
    index  index.html index.htm;

#反向代理路径
    proxy_pass http://myserver;

#反向代理的超时时间
    proxy_connect_timeout 10;

    proxy_redirect default;
}
#普通匹配
location /images/ {
    root images ;
}

# 反正则匹配
location ^~ /images/jpg/ { # 匹配任何以 /images/jpg/ 开头的任何查询并且停止搜索。任何正则表
达式将不会被测试。
    root images/jpg/ ;
}
#正则匹配
location ~*.(gif|jpg|jpeg)$ {

```

```
#所有静态文件直接读取硬盘
    root pic ;

    #expires 定义用户浏览器缓存的时间为 3 天，如果静态页面不常更新，可以设置更长，这样可以节省带宽和缓解服务器的压力
    expires 3d; #缓存 3 天
}

#error_page 404 /404.html;

# redirect server error pages to the static page /50x.html
#
error_page 500 502 503 504 /50x.html;
location = /50x.html {
    root html;
}
}
```

7、Nginx 重要知识点

Nginx 反向代理

Nginx 负载均衡

Nginx 动静分离

Nginx 集群

Nginx 原理及优化配置：